



AI ENGINEERING & AGENT DEVELOPMENT – IMPLEMENTATION INSTRUCTIONS

Files & Docs Module + AI-Assisted Post Creation

1. PURPOSE

This document provides implementation guidance for building an **AI-enabled Files & Docs system and Smart Post Creation experience** based on an existing UI/UX design.

You are expected to translate the design into a **functional backend + AI workflow system** that integrates:

- File storage and moderation states
- Media retrieval and selection
- AI-assisted content generation
- Smart recommendations

2. SYSTEM CONTEXT

The platform includes a **Files & Docs module**, which acts as a **central media library** where users can:

- Upload files (images, videos, audio, documents)
- View and manage uploaded content
- Select media when creating posts

Backend Workflow (Already Implemented on AWS, you will need to orchestrate in your development backend and simulate the various instances)

- File uploads are processed via AWS (S3, Lambda, Step Functions)
- Each file goes through a **moderation scan pipeline**
- Files are:
 - o Approved → Available in the module
 - o Rejected → Not visible / flagged

3. YOUR RESPONSIBILITY

You are responsible for implementing:

A. Backend + AI Logic

- AI-assisted post creation
- Media recommendation system
- Integration with Files & Docs module

B. System Integration

- UI → API → AI → Database → Response flow
- Ensure all features are usable in a real product environment

4. CORE FUNCTIONAL REQUIREMENTS

4.1 Files & Docs Module (Reference Integration)

You should assume the UI already supports:

- Categories:
 - o All Files
 - o Images
 - o Videos
 - o Audio
 - o Documents
- File metadata:
 - o Name, type, size, upload date
- Upload states:
 - o Upload initiated
 - o Scan in progress
 - o Approved
 - o Rejected

Your Role:

- Ensure backend/API supports these states
- Ensure only **approved files** are retrievable for use

4.2 Upload & Moderation Awareness

When files are uploaded:

- The system must track:
 - o Upload status
 - o Scan progress
 - o Final moderation result

Expected Logic:

- Store file status in database (e.g., MongoDB)
- Expose API endpoints to:
 - o Fetch file list by status
 - o Update status based on AWS workflow

4.3 AI-Assisted Post Creation

When a user is creating a post:

The system should:

1. Accept:
 - o Topic / prompt / partial text
2. Generate:
 - o Suggested post content
 - o Improved/rephrased versions
 - o Alternative variations
3. Support:
 - o Regenerate
 - o Edit
 - o Accept output

4.4 Smart Media Recommendations

When a user is writing a post or attaching media:

The system should:

- Recommend relevant files from the Files & Docs module

Recommendation Inputs:

- Post content (text context)



- File metadata (type, name, tags if available)
- User-specific data (optional bonus)

Expected Output:

- Ranked list of relevant files

4.5 Asset Picker Integration

When a user clicks “Attach Media”:

- DO NOT allow direct local uploads
- Instead:
 - o Open Files & Docs asset picker
 - o Show categorized files
 - o Include AI-recommended assets

4.6 AI Recommendation Enhancements

The AI should also support:

- Hashtag suggestions
- Content improvement suggestions
- Related post ideas

5. TECHNICAL EXPECTATIONS

Backend

- Node.js (Express)
- REST API design
- MongoDB (file metadata, user context, logs)

AI Layer

- OpenAI (or equivalent LLM API)
- Prompt orchestration
- Context injection (post + file metadata)

Retrieval Logic (Important)

At minimum, implement:

- Keyword or semantic matching between:
 - o Post content
 - o File metadata

Bonus:

- Embeddings / vector-based retrieval

6. EXPECTED SYSTEM FLOW

You should implement a structured pipeline:

```
User Input (Post / Action)
      ↓
API Endpoint (Node.js)
      ↓
AI Processing (Prompt + Context)
      ↓
Retrieval Layer (Files & Docs)
      ↓
Response (Suggestions / Content)
      ↓
Frontend Consumption {Reactjs ,Typescript, Tanstack-Router and Zustand for
State Management }
```

7. API REQUIREMENTS

You should implement endpoints such as:

AI Features

- POST /ai/generate-post
- POST /ai/recommend-media
- POST /ai/suggest-hashtags

Files Module

- GET /files
- GET /files?type=image
- GET /files?status=approved



8. EDGE CASES TO HANDLE

- Empty user input
- No relevant files found
- AI failure / timeout
- Rejected files appearing in results (must NOT happen)
- Slow responses (consider fallback or loading states)

9. DELIVERABLES

You are expected to submit:

1. Codebase

- Working backend APIs
- AI integration logic

2. System Design Explanation

- Architecture overview
- AI workflow explanation
- Retrieval approach

3. Sample Outputs

- Example prompts → generated posts
- Example recommendations

IMPLEMENTATION QUALITY EXPECTATIONS

Your submission must not be treated as a prototype-only exercise. Even where simplified, the implementation should show how you think about building AI-enabled functionality in a maintainable and real-world-ready way.

Your implementation should therefore demonstrate the following:

A. Clear Technical Separation

Your code should show sensible separation between:

- route/controller logic
- AI generation logic

- recommendation/retrieval logic
- database interaction
- validation and error handling
- shared helpers/types where relevant

Avoid placing all logic in a single endpoint file.

B. Safe Retrieval Boundaries

You must ensure that only approved files are returned for recommendation or attachment use. Rejected or non-approved files must never appear in recommendation results or asset picker flows.

C. Operational Awareness

Your implementation should show basic operational readiness thinking, including:

- how failures are surfaced
- how timeouts are handled
- how fallback responses are returned
- how important events or failures could be logged for later debugging or review

D. Maintainability

Your code and documentation should be understandable by another engineer. Assume that someone else may need to continue your work later.

E. Product Behaviour Under Weak Conditions

You should show how the feature behaves when:

- the user input is too weak
- no file is relevant enough to recommend
- AI output quality is weak
- the AI service is unavailable
- the retrieval layer succeeds but the AI layer fails
- the AI layer succeeds but no file is safe or eligible to recommend

The goal is not only to produce outputs, but to show product-safe and user-clear behaviour.

10. SUCCESS CRITERIA

Your solution will be evaluated based on:

- Structured AI workflow rather than raw API calling only
- Relevance and usefulness of generated outputs
- Quality and safety of media recommendations
- Clean backend architecture and separation of concerns
- Respect for file moderation / approval states
- Real-world usability considerations with frontend integration
- Clarity of fallback handling and failure behaviour
- Maintainability and readability of the codebase
- Quality of documentation and handover explanation
- Evidence of product thinking, not just technical implementation

A stronger submission will show:

- thoughtful output shaping
- sensible ranking/recommendation behaviour
- practical handling of weak input and no-result scenarios
- awareness of latency, reliability, and operational support needs
- readiness for future continuation by another engineer

11. IMPORTANT NOTES

- Focus on **practical implementation**, not theory
- Prioritize **clarity, reliability, and usability**
- Avoid overly complex solutions if not necessary
- Demonstrate **product thinking, not just coding**